

IIT Bhubaneswar - Road Safety Classification using Multimodal Deep Learning Resnet-50 and DenseNet-121 Models



```
import numpy as np
import pandas as pd
import os
```

```
df = pd.read_csv('/kaggle/input/road-safety-analysis-iit-
bhubaneswar/IIT_BBS_Campus/data_cleaned.csv')
```

```
df.head()
```

	Name	Time	Human_Activity	Luminosity	Terrain	\
0	IMG_1854.jpg	Night	0	Well_Lit	Road	
1	IMG_1855.jpg	Night	0	Well_Lit	Road	
2	IMG_1856.jpg	Morning	0	Daylight	Road	
3	IMG_1857.jpg	Afternoon	0	Daylight	Road	
4	IMG_1858.jpg	Afternoon	0	Daylight	Muddy	

	Presence_of_building	Mechanical_Activity	Nearby_vegetation	Safety
0	0	0	Low	Safe
1	0	0	Low	Safe
2	1	0	Low	Safe
3	1	0	Low	Safe
4	0	0	High	Risky

```
df.tail()
```

	Name	Time	Human_Activity	Luminosity	Terrain	\
907	20251116_142845.jpg	Afternoon	0	Daylight	Muddy	
908	20251116_142902.jpg	Afternoon	0	Daylight	Rough	
909	20251116_142939.jpg	Afternoon	0	Daylight	Rough	
910	20251116_143135.jpg	Afternoon	0	Daylight	Muddy	
911	20251116_143215.jpg	Afternoon	1	Daylight	Muddy	

	Presence_of_building	Mechanical_Activity	Nearby_vegetation	Safety
907	1	0	High	Risky
908	0	0	High	Risky
909	0	0	High	Risky
910	0	0	High	Dangerous
911	0	0	High	Risky

```
df.shape
```

```
(912, 9)
```

```
df.columns
```

```
Index(['Name', 'Time', 'Human_Activity', 'Luminosity', 'Terrain',
      'Presence_of_building', 'Mechanical_Activity', 'Nearby_vegetation',
      'Safety'],
      dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
Name          0
Time          0
Human_Activity 0
Luminosity    0
Terrain       0
Presence_of_building 0
Mechanical_Activity 0
Nearby_vegetation 0
Safety        0
dtype: int64
```

```
df.nunique()
```

```
Name          912
Time           4
Human_Activity 2
Luminosity     5
Terrain        3
Presence_of_building 2
Mechanical_Activity 2
Nearby_vegetation 3
Safety         3
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 912 entries, 0 to 911
```

```
Data columns (total 9 columns):
```

```
#    Column          Non-Null Count  Dtype
```

```

---
0  Name          912 non-null  object
1  Time          912 non-null  object
2  Human_Activity 912 non-null  int64
3  Luminosity    912 non-null  object
4  Terrain        912 non-null  object
5  Presence_of_building 912 non-null int64
6  Mechanical_Activity 912 non-null int64
7  Nearby_vegetation 912 non-null object
8  Safety         912 non-null  object
dtypes: int64(3), object(6)
memory usage: 64.3+ KB

df['Safety'].unique()

array(['Safe', 'Risky', 'Dangerous'], dtype=object)

df['Safety'].value_counts()

Safety
Safe      483
Dangerous 232
Risky     197
Name: count, dtype: int64

base_dir = "/kaggle/input/road-safety-analysis-iit-
bhubaneswar/IIT_BBS_Campus/Images"

def find_real_image_path(short_name):
    short_name = short_name.strip()
    for file in os.listdir(base_dir):
        if file.endswith(short_name):
            return os.path.join(base_dir, file)
    raise FileNotFoundError(f"Image not found for {short_name}")

df['image_path'] = df['Name'].apply(find_real_image_path)

new_df = df[[
    'image_path', 'Time', 'Human_Activity', 'Luminosity',
    'Terrain', 'Presence_of_building', 'Mechanical_Activity',
    'Nearby_vegetation', 'Safety'
]].copy()

new_df = new_df.rename(columns={'Safety': 'label'})
new_df['label'] = new_df['label'].map({'Safe': 0, 'Risky': 1, 'Dangerous':
2})

new_df = pd.get_dummies(new_df, columns=['Time', 'Luminosity', 'Terrain',
'Nearby_vegetation'], dtype=int)

cols = ['image_path', 'label'] + [c for c in new_df.columns if c not in
['image_path', 'label']]
new_df = new_df[cols].reset_index(drop=True)

```

new_df

	image_path	label	Human_Activity
\			
0	/kaggle/input/road-safety-analysis-iit-bhubane...	0	0
1	/kaggle/input/road-safety-analysis-iit-bhubane...	0	0
2	/kaggle/input/road-safety-analysis-iit-bhubane...	0	0
3	/kaggle/input/road-safety-analysis-iit-bhubane...	0	0
4	/kaggle/input/road-safety-analysis-iit-bhubane...	1	0
..	...	...	...
907	/kaggle/input/road-safety-analysis-iit-bhubane...	1	0
908	/kaggle/input/road-safety-analysis-iit-bhubane...	1	0
909	/kaggle/input/road-safety-analysis-iit-bhubane...	1	0
910	/kaggle/input/road-safety-analysis-iit-bhubane...	2	0
911	/kaggle/input/road-safety-analysis-iit-bhubane...	1	1

	Presence_of_building	Mechanical_Activity	Time_Afternoon	Time_Evening
\				
0	0	0	0	0
1	0	0	0	0
2	1	0	0	0
3	1	0	1	0
4	0	0	1	0
..	...	...	...	...
907	1	0	1	0
908	0	0	1	0
909	0	0	1	0
910	0	0	1	0
911	0	0	1	0

	Time_Morning	Time_Night	Luminosity_Dark	Luminosity_Daylight	\
0	0	1	0	0	
1	0	1	0	0	
2	1	0	0	1	
3	0	0	0	1	
4	0	0	0	1	
..	...	...	...	...	
907	0	0	0	1	
908	0	0	0	1	
909	0	0	0	1	
910	0	0	0	1	
911	0	0	0	1	

	Luminosity_Mildly_Lit	Luminosity_Well_Lit	Luminosity_Well_lit	\
0	0	1	0	
1	0	1	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
..	...	...	...	
907	0	0	0	
908	0	0	0	

909	0	0	0
910	0	0	0
911	0	0	0

	Terrain_Muddy	Terrain_Road	Terrain_Rough	Nearby_vegetation_High	\
0	0	1	0	0	
1	0	1	0	0	
2	0	1	0	0	
3	0	1	0	0	
4	1	0	0	1	
..	...	...	...	...	
907	1	0	0	1	
908	0	0	1	1	
909	0	0	1	1	
910	1	0	0	1	
911	1	0	0	1	

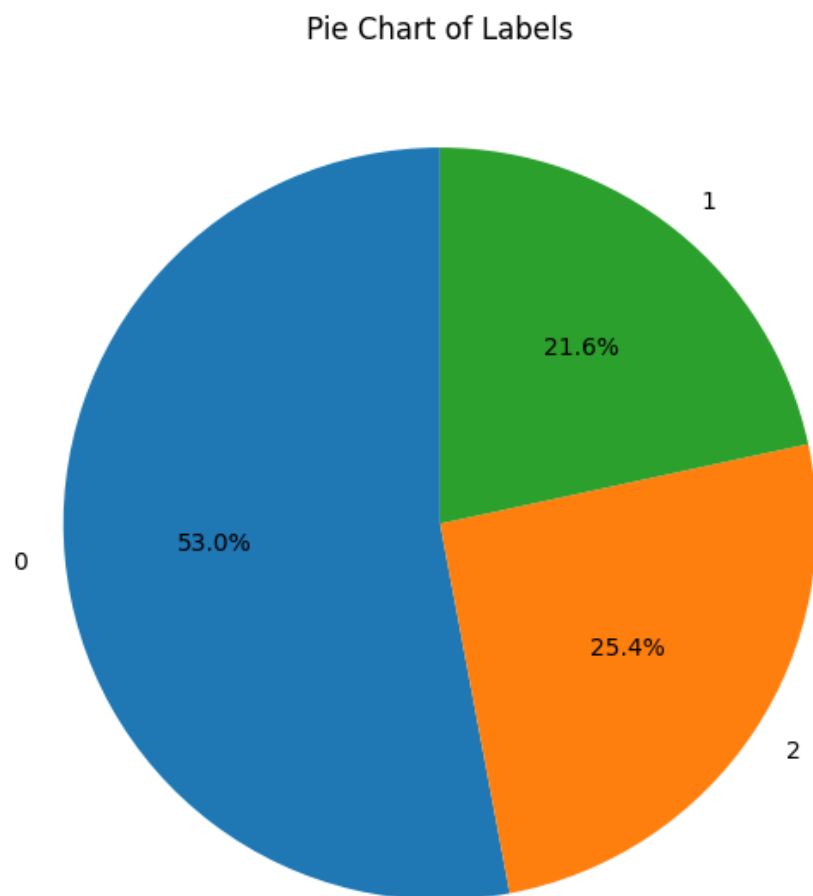
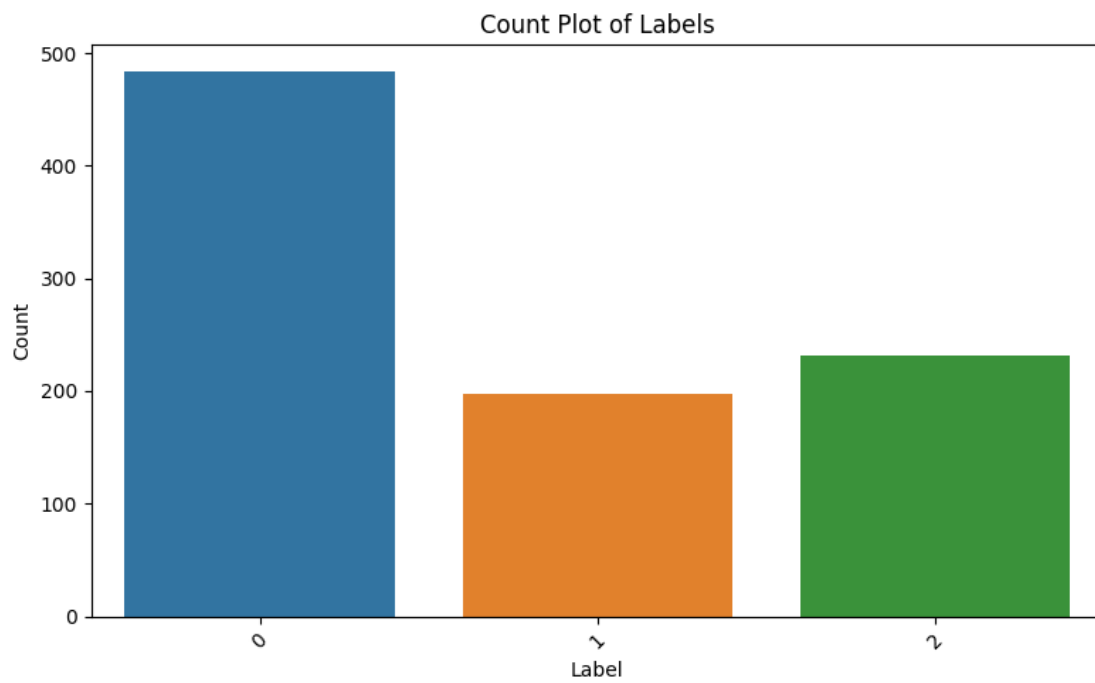
	Nearby_vegetation_Low	Nearby_vegetation_Medium
0	1	0
1	1	0
2	1	0
3	1	0
4	0	0
..	...	...
907	0	0
908	0	0
909	0	0
910	0	0
911	0	0

[912 rows x 20 columns]

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(8,5))
sns.countplot(x=new_df['label'])
plt.title("Count Plot of Labels")
plt.xlabel("Label")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(6,6))
new_df['label'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90)
plt.title("Pie Chart of Labels")
plt.ylabel("")
plt.tight_layout()
plt.show()
```



```

import random
from PIL import Image

random.seed(42)

fig, axes = plt.subplots(3, 5, figsize=(20, 12))
fig.suptitle('5 Sample Images Per Class', fontsize=22, weight='bold')

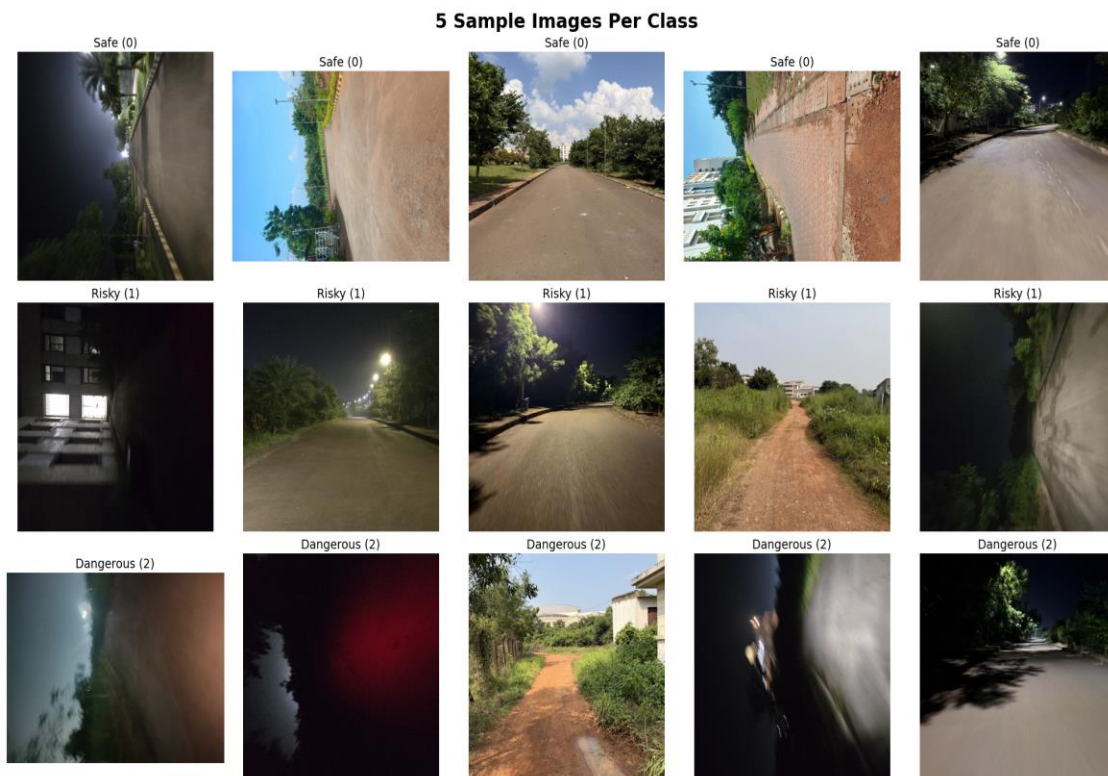
for label_idx, label_name in enumerate(['Safe (0)', 'Risky (1)', 'Dangerous (2)']):
    label_df = new_df[new_df['label'] == label_idx]

    valid_rows = []
    for _, row in label_df.sample(frac=1, random_state=42).iterrows():
        if os.path.exists(row['image_path']):
            valid_rows.append(row)
        if len(valid_rows) == 5:
            break

    for i, row in enumerate(valid_rows):
        img = Image.open(row['image_path'])
        axes[label_idx, i].imshow(img)
        axes[label_idx, i].set_title(label_name, fontsize=14)
        axes[label_idx, i].axis('off')

plt.tight_layout()
plt.subplots_adjust(top=0.93)
plt.show()

```



```

import warnings
warnings.filterwarnings('ignore')

from sklearn.utils import resample

print("Original distribution:")
print(new_df['label'].value_counts().sort_index())

df_safe      = new_df[new_df['label'] == 0]
df_risky     = new_df[new_df['label'] == 1]
df_dangerous = new_df[new_df['label'] == 2]

target_count = new_df['label'].value_counts().max() + 50

df_safe_res   = resample(df_safe,      replace=True,
n_samples=target_count, random_state=42)
df_risky_res  = resample(df_risky,     replace=True,
n_samples=target_count, random_state=42)
df_dangerous_res = resample(df_dangerous, replace=True,
n_samples=target_count, random_state=42)
new_df_balanced = pd.concat([df_safe_res, df_risky_res, df_dangerous_res])

new_df_balanced = new_df_balanced.sample(frac=1,
random_state=42).reset_index(drop=True)

print("\nBalanced distribution:")
print(new_df_balanced['label'].value_counts().sort_index())

```

```

Original distribution:
label
0      483
1      197
2      232
Name: count, dtype: int64

```

```

Balanced distribution:
label
0      533
1      533
2      533
Name: count, dtype: int64

```

```
df = new_df_balanced
```

```
df
```

```

              image_path  label  \
0  /kaggle/input/road-safety-analysis-iit-bhubane...      1
1  /kaggle/input/road-safety-analysis-iit-bhubane...      0
2  /kaggle/input/road-safety-analysis-iit-bhubane...      0
3  /kaggle/input/road-safety-analysis-iit-bhubane...      1
4  /kaggle/input/road-safety-analysis-iit-bhubane...      2
...                  ...      ...

```


1594	/kaggle/input/road-safety-analysis-iit-bhubane...	2
1595	/kaggle/input/road-safety-analysis-iit-bhubane...	2
1596	/kaggle/input/road-safety-analysis-iit-bhubane...	1
1597	/kaggle/input/road-safety-analysis-iit-bhubane...	2
1598	/kaggle/input/road-safety-analysis-iit-bhubane...	2

	Human_Activity	Presence_of_building	Mechanical_Activity	\
0	0	1	0	
1	0	1	0	
2	1	1	1	
3	1	1	0	
4	0	0	0	
...	...	...	...	
1594	0	1	0	
1595	0	0	0	
1596	0	0	0	
1597	0	0	0	
1598	0	1	0	

	Time_Afternoon	Time_Evening	Time_Morning	Time_Night	Luminosity_Dark
\					
0	0	0	0	1	1
1	1	0	0	0	0
2	0	0	0	1	0
3	0	0	0	1	0
4	0	1	0	0	0
...	...	...	...	...	...
1594	0	0	0	1	0
1595	0	0	0	1	1
1596	1	0	0	0	0
1597	0	0	0	1	0
1598	0	0	0	1	0

	Luminosity_Daylight	Luminosity_Mildly_Lit	Luminosity_Well_Lit	\
0	0	0	0	
1	1	0	0	
2	0	0	1	
3	0	1	0	
4	0	1	0	
...	...	...	...	
1594	0	1	0	
1595	0	0	0	
1596	1	0	0	
1597	0	1	0	
1598	0	0	1	

	Luminosity_Well_lit	Terrain_Muddy	Terrain_Road	Terrain_Rough	\
0	0	1	0	0	
1	0	0	1	0	
2	0	0	1	0	
3	0	0	1	0	
4	0	0	0	1	

...	...	...	...	...
1594	0	1	0	0
1595	0	0	1	0
1596	0	0	0	1
1597	0	0	0	1
1598	0	0	1	0

	Nearby_vegetation_High	Nearby_vegetation_Low	Nearby_vegetation_Medium
0	0	1	0
1	0	0	1
2	0	0	1
3	0	1	0
4	1	0	0
...	...	...	...
1594	1	0	0
1595	1	0	0
1596	0	0	1
1597	1	0	0
1598	1	0	0

[1599 rows x 20 columns]

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import models, transforms
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
from PIL import Image
```

```
torch.manual_seed(42)
np.random.seed(42)
```

```
class RoadSafetyDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(row['image_path']).convert('RGB')
        if self.transform:
            img = self.transform(img)
        label = torch.tensor(row['label'], dtype=torch.long)
        tabular = torch.tensor(row.drop(['image_path',
'label']).values.astype('float32'))
        return img, tabular, label
```

```

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

```

```

full_dataset = RoadSafetyDataset(df, transform=transform)

```

```

train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size,
val_size])

```

```

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader    = DataLoader(val_dataset,    batch_size=32, shuffle=False)

```

```

class MultimodalModel(nn.Module):
    def __init__(self, tabular_dim, num_classes=3):
        super().__init__()
        self.cnn = models.resnet50(pretrained=True)
        self.cnn.fc = nn.Identity()

        self.tabular_mlp = nn.Sequential(
            nn.Linear(tabular_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.ReLU()
        )

        self.classifier = nn.Sequential(
            nn.Linear(2048 + 128, 512),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(512, num_classes)
        )

    def forward(self, img, tab):
        img_feat = self.cnn(img)
        tab_feat = self.tabular_mlp(tab)
        combined = torch.cat([img_feat, tab_feat], dim=1)
        return self.classifier(combined)

```

```

tabular_dim = len(df.columns) - 2
model = MultimodalModel(tabular_dim=tabular_dim, num_classes=3)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

```

```

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0003)

```

```

history = {'train_loss': [], 'val_loss': [], 'val_acc': []}
all_preds = []
all_labels = []

print("Training started...")
for epoch in range(5):
    model.train()
    running_loss = 0.0
    for imgs, tabs, labels in train_loader:
        imgs, tabs, labels = imgs.to(device), tabs.to(device),
labels.to(device)

        optimizer.zero_grad()
        outputs = model(imgs, tabs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for imgs, tabs, labels in val_loader:
            imgs, tabs, labels = imgs.to(device), tabs.to(device),
labels.to(device)
            outputs = model(imgs, tabs)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    train_loss = running_loss / len(train_loader)
    val_loss = val_loss / len(val_loader)
    val_acc = 100 * correct / total

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    print(f"Epoch {epoch+1}/5 - Train Loss: {train_loss:.4f} | Val Loss:
{val_loss:.4f} | Val Acc: {val_acc:.2f}%")

print("\n" + "="*60)
print("FINAL VALIDATION ACCURACY:", history['val_acc'][-1])

```

```

print("="*60)

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(history['train_loss'], label='Train Loss')
plt.plot(history['val_loss'], label='Val Loss')
plt.title('Loss')
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(history['val_acc'], label='Validation Accuracy', color='green')
plt.title('Validation Accuracy')
plt.ylabel('Accuracy %')
plt.legend()

plt.subplot(1, 3, 3)
cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Safe', 'Risky', 'Dangerous'],
            yticklabels=['Safe', 'Risky', 'Dangerous'])
plt.title('Confusion Matrix')
plt.ylabel('True')
plt.xlabel('Predicted')

plt.tight_layout()
plt.show()

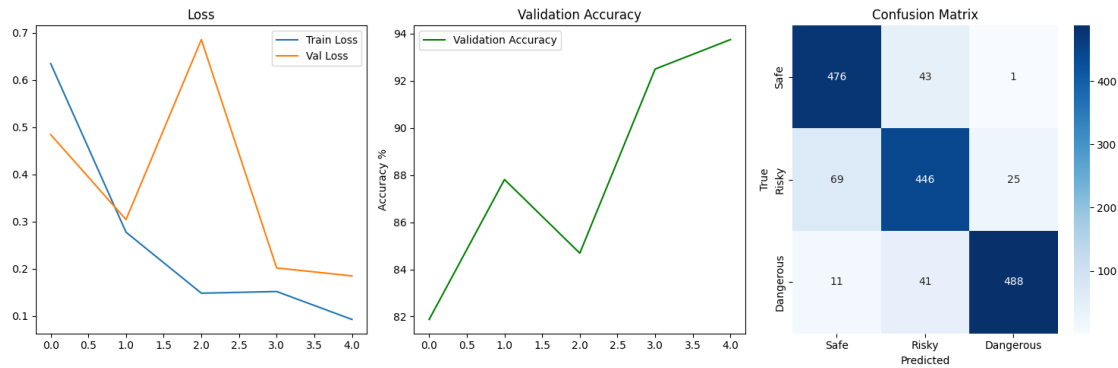
print("\nClassification Report:")
print(classification_report(all_labels, all_preds,
                           target_names=['Safe', 'Risky', 'Dangerous'],
                           digits=4))

Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to
/root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 187MB/s]

Training started...
Epoch 1/5 - Train Loss: 0.6347 | Val Loss: 0.4842 | Val Acc: 81.88%
Epoch 2/5 - Train Loss: 0.2773 | Val Loss: 0.3041 | Val Acc: 87.81%
Epoch 3/5 - Train Loss: 0.1480 | Val Loss: 0.6857 | Val Acc: 84.69%
Epoch 4/5 - Train Loss: 0.1516 | Val Loss: 0.2016 | Val Acc: 92.50%
Epoch 5/5 - Train Loss: 0.0924 | Val Loss: 0.1847 | Val Acc: 93.75%

=====
FINAL VALIDATION ACCURACY: 93.75
=====

```



Classification Report:

	precision	recall	f1-score	support
Safe	0.8561	0.9154	0.8848	520
Risky	0.8415	0.8259	0.8336	540
Dangerous	0.9494	0.9037	0.9260	540
accuracy			0.8812	1600
macro avg	0.8823	0.8817	0.8815	1600
weighted avg	0.8827	0.8812	0.8814	1600

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
from torchvision import models, transforms
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from PIL import Image

```

```

torch.manual_seed(42)
np.random.seed(42)

```

```

class RoadSafetyDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(row['image_path']).convert('RGB')
        if self.transform:
            img = self.transform(img)
        label = torch.tensor(row['label'], dtype=torch.long)
        tabular = torch.tensor(row.drop(['image_path',

```

```

'label']).values.astype('float32'))
    return img, tabular, label

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])
])

full_dataset = RoadSafetyDataset(df, transform=transform)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size,
val_size])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

class MultimodalModel(nn.Module):
    def __init__(self, tabular_dim, num_classes=3):
        super().__init__()
        densenet = models.densenet121(pretrained=True)
        self.cnn = densenet.features
        self.cnn.avgpool = nn.AdaptiveAvgPool2d(1)
        self.cnn = nn.Sequential(self.cnn, self.cnn.avgpool)

        self.tabular_mlp = nn.Sequential(
            nn.Linear(tabular_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 128),
            nn.ReLU()
        )

        self.classifier = nn.Sequential(
            nn.Linear(1024 + 128, 512),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(512, num_classes)
        )

    def forward(self, img, tab):
        img_feat = self.cnn(img).flatten(1)
        tab_feat = self.tabular_mlp(tab)
        combined = torch.cat([img_feat, tab_feat], dim=1)
        return self.classifier(combined)

tabular_dim = len(df.columns) - 2
model = MultimodalModel(tabular_dim=tabular_dim, num_classes=3)

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.0003)

history = {'train_loss': [], 'val_loss': [], 'val_acc': []}
all_preds = []
all_labels = []

print("Training started with DenseNet-121 (light & fast)...")
for epoch in range(5):
    model.train()
    running_loss = 0.0
    for imgs, tabs, labels in train_loader:
        imgs, tabs, labels = imgs.to(device), tabs.to(device),
labels.to(device)

        optimizer.zero_grad()
        outputs = model(imgs, tabs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for imgs, tabs, labels in val_loader:
            imgs, tabs, labels = imgs.to(device), tabs.to(device),
labels.to(device)

            outputs = model(imgs, tabs)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

    train_loss = running_loss / len(train_loader)
    val_loss = val_loss / len(val_loader)
    val_acc = 100 * correct / total

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

```



```

        print(f"Epoch {epoch+1}/5 - Train Loss: {train_loss:.4f} | Val Loss:
{val_loss:.4f} | Val Acc: {val_acc:.2f}%")

print("\n" + "="*60)
print("FINAL VALIDATION ACCURACY:", history['val_acc'][-1])
print("="*60)

```

```
plt.figure(figsize=(15, 5))
```

```

plt.subplot(1, 3, 1)
plt.plot(history['train_loss'], label='Train Loss')
plt.plot(history['val_loss'], label='Val Loss')
plt.title('Loss')
plt.legend()

```

```

plt.subplot(1, 3, 2)
plt.plot(history['val_acc'], label='Validation Accuracy', color='green')
plt.title('Validation Accuracy')
plt.ylabel('Accuracy %')
plt.legend()

```

```

plt.subplot(1, 3, 3)
cm = confusion_matrix(all_labels, all_preds)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Safe', 'Risky', 'Dangerous'],
            yticklabels=['Safe', 'Risky', 'Dangerous'])
plt.title('Confusion Matrix')
plt.ylabel('True')
plt.xlabel('Predicted')

```

```

plt.tight_layout()
plt.show()

```

```

print("\nClassification Report:")
print(classification_report(all_labels, all_preds,
                           target_names=['Safe', 'Risky', 'Dangerous'],
                           digits=4))

```

```

Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth"
to /root/.cache/torch/hub/checkpoints/densenet121-a639ec97.pth
100%|██████████| 30.8M/30.8M [00:00<00:00, 148MB/s]

```

```
Training started with DenseNet-121 (light & fast)...
```

```

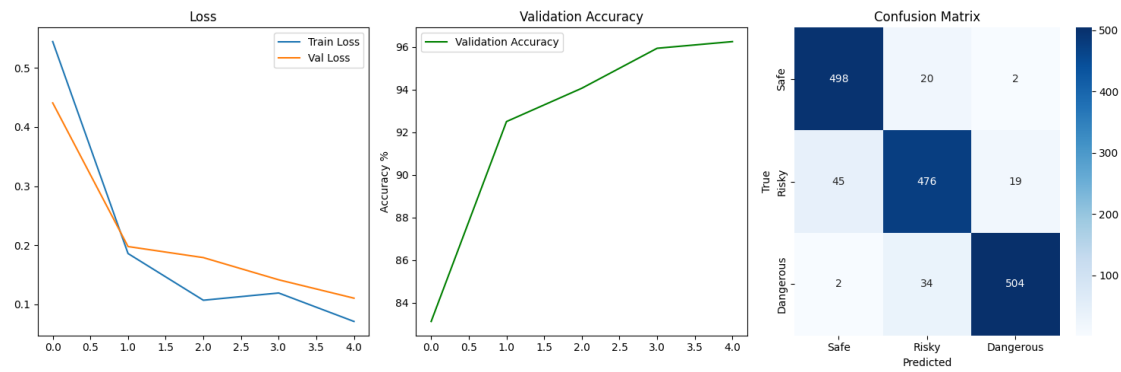
Epoch 1/5 - Train Loss: 0.5446 | Val Loss: 0.4409 | Val Acc: 83.12%
Epoch 2/5 - Train Loss: 0.1860 | Val Loss: 0.1978 | Val Acc: 92.50%
Epoch 3/5 - Train Loss: 0.1070 | Val Loss: 0.1791 | Val Acc: 94.06%
Epoch 4/5 - Train Loss: 0.1192 | Val Loss: 0.1416 | Val Acc: 95.94%
Epoch 5/5 - Train Loss: 0.0711 | Val Loss: 0.1105 | Val Acc: 96.25%

```

```

=====
FINAL VALIDATION ACCURACY: 96.25
=====

```



Classification Report:

	precision	recall	f1-score	support
Safe	0.9138	0.9577	0.9352	520
Risky	0.8981	0.8815	0.8897	540
Dangerous	0.9600	0.9333	0.9465	540
accuracy			0.9237	1600
macro avg	0.9240	0.9242	0.9238	1600
weighted avg	0.9241	0.9237	0.9237	1600